Working with Simple Queues

Operations on Simple Queues

So, we've learned what a Queue is and the key terminologies related to it. We're now ready to roll up our sleeves and start working with Simple Queues. Remember our coffee shop line? Let's think of our Simple Queue in the same way.

The primary operations that we can perform on a Simple Queue are: Enqueue (add an element to the end of the Queue), and Dequeue (remove an element from the front). There's also Peek or Front (get the value of the front item without removing it), and IsEmpty (check if the Queue is empty).

Enqueue Operation

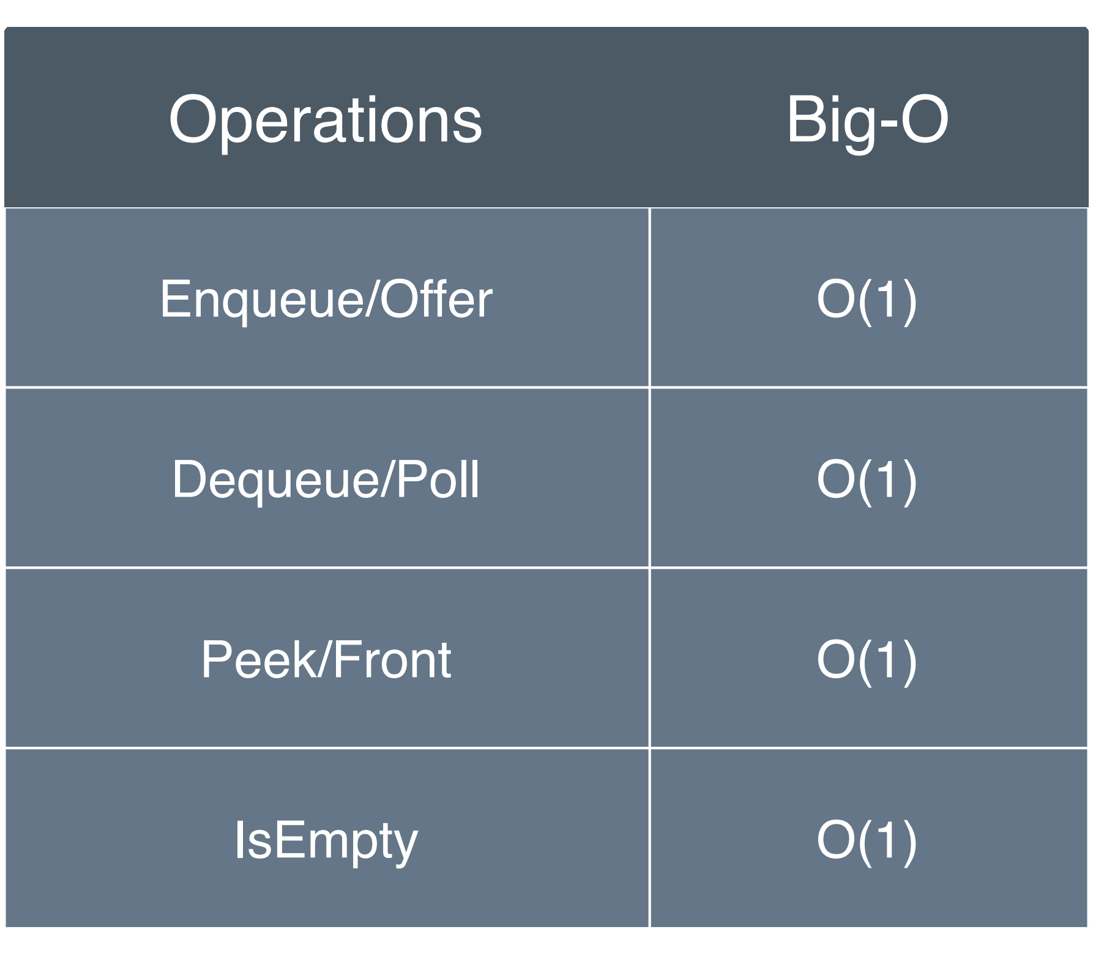
The Enqueue operation is about adding an element at the rear of the Queue. The step is simple: we take the item we want to add, go to the end of the line (rear of the Queue), and place it there. But wait, how do we know if there's enough space in our Queue to add a new element? Good question! Before we add a new item, we must check if our Queue is already full. If it's full, we can't add a new item - we call this situation Queue Overflow.

Dequeue Operation

The Dequeue operation is about removing an element from the front of the Queue. So, we go to the front of our Queue, serve the first customer (remove the first element), and adjust our front to point to the next customer. But here too, we need to be careful. Before we remove an item, we must check if there are any items to remove! If the Queue is empty, we can't remove anything - we call this situation Queue Underflow.

Peek and IsEmpty Operations

Peek (or Front) and IsEmpty are straightforward operations. Peek lets us see the first element in the Queue (like calling the next customer without serving them), and IsEmpty lets us know if our Queue is empty (like checking if there are any customers left to serve).



Operations	Big-O
Enqueue/Offer	O(1)
Dequeue/Poll	O(1)
Peek/Front	O(1)
IsEmpty	O(1)

Time Complexity of Queue Operations

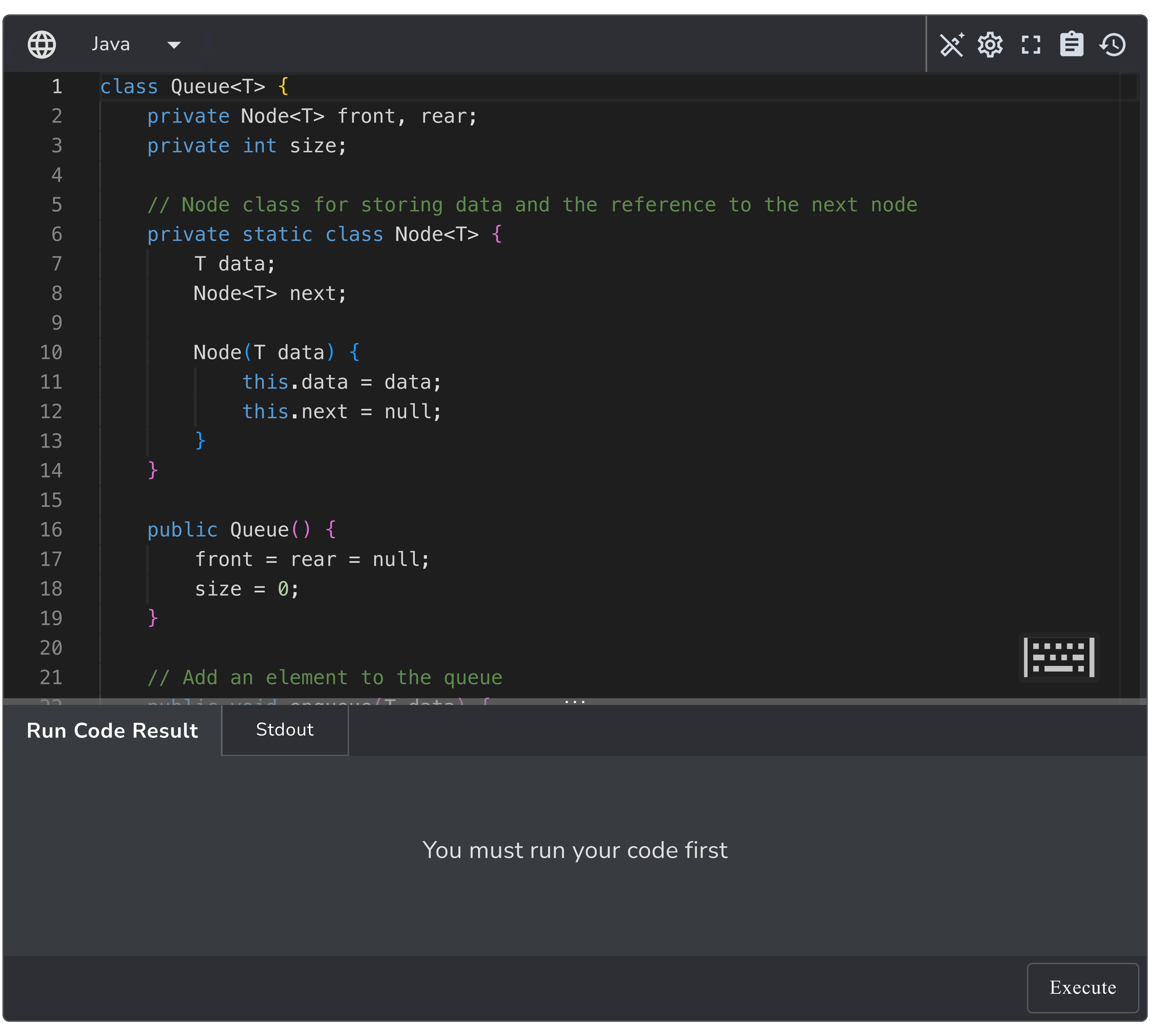
Simple Queue in Action

In this section, we're going to look at some examples of the abovementioned operations and how they work in a program.

But before we jump into coding, let's consider how we're going to structure our Queue. A simple way is to implement a queue using a LinkedList. We can use a Node structure to represent each element of the Queue. Each Node will have a variable to store the data and a Node pointer which will be pointing to the next element of the queue. In the Queue class, we will have two pointers: one to the front Node of the Queue, and one to the rear Node. Here is the description of each element of the queue class:

1. **Node Class:** This is an inner class used to represent each element in the queue. Each node contains the data and a reference to the next node in the list.
2. **Queue Class Attributes:**
   - `front`: A reference to the first node in the queue.
   - `rear`: A reference to the last node in the queue.
   - `size`: An integer tracking the number of elements in the queue.
3. **Constructor:**
   - The constructor initializes an empty queue where both `front` and `rear` are set to `null`, and `size` is set to 0.
4. **Enqueue Operation** (`enqueue` method):
   - This method adds an element to the end of the queue.
   - A new node is created with the given data.
   - If the queue is empty (`rear == null`), both `front` and `rear` point to the new node.
   - If the queue is not empty, the new node is added after `rear`, and then `rear` is updated to point to this new node.
   - The `size` of the queue is incremented.
5. **Dequeue Operation** (`dequeue` method):
   - This method removes an element from the front of the queue.
   - If the queue is empty (`front == null`), the method returns `null`.
   - The data from the `front` node is saved.
   - `front` is updated to point to the next node in the queue.
   - If the queue becomes empty after this operation (`front == null`), `rear` is also set to `null`.
   - The `size` of the queue is decremented.
   - The method returns the saved data.
6. **Peek Operation** (`peek` method):
   - This method returns the data from the front node without removing it.
   - If the queue is empty (`front == null`), it returns `null`.
   - Otherwise, it returns the data of the `front` node.
7. **Utility Methods:**
   - `isEmpty`: Checks if the queue is empty (i.e., `size == 0`).
   - `size`: Returns the number of elements in the queue.

Here's a program to implement the queue. This implementation of a queue is a classic example of a FIFO (First-In-First-Out) data structure, where elements are added to the rear and removed from the front, ensuring that the order in which elements are added is the order in which they are removed.



```java
class Queue<T> {
    private Node<T> front, rear;
    private int size;

    // Node class for storing data and the reference to the next node
    private static class Node<T> {
        T data;
        Node<T> next;

        Node(T data) {
            this.data = data;
            this.next = null;
        }
    }

    public Queue() {
        front = rear = null;
        size = 0;
    }

    // Add an element to the queue
```

You must run your code first

Queue Implementation

Here is how different languages have implemented Queue:

Language	API
Java	java.util.Queue
Python	queue
C++	std::queue
JavaScript	Implemented through Array

← Previous
Introduction To Queues

Next →
Diving Deeper – Circular Queues And Deques

☐ Mark as Completed